COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	7/16/26	**DATE SUBMITTED:**	7/20/26
NAME:	Vince Andrew Urbano	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2 **Arrays, Pointers and Dynamic Memory Allocations**

1. PROCEDURE OUTPUT

ILO-A:

Screenshot

ilo-a.cpp > main()

```cpp
#include <iostream>
#include <string.h>
class Student{
  private:
    std::string studentName;
    int studentAge;
  public:
  //constructor
    Student(std::string newName = "John Doe", int newAge=18){
      studentName = std::move(newName);
      studentAge = newAge;
      std::cout << "Constructor called" << std::endl;
    };
    ~Student(){
      std::cout << "Destructor called" <<std::endl;
    }
    //Copy Constructor
    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }
    //Display Attributes
    void printDetails(){
        std::cout<<this->studentName << " " << this->studentAge << std::endl;
    }
};
int main() {
    Student student1("Vince", 19);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;

}
```

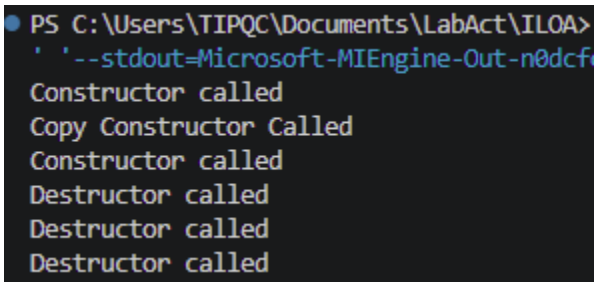
Output	
Observation	In these lines of codes, we can see that no values were printed out. This is because there 's no specific code that tells the program to print the values. Moving on, "Constructor Called" was printed out first because it was called on line 29 inside the int main(). On line 30, we can see that the student2 object just copied the value of student1, this caused the Copy constructor to be called. We can also see that the constructor was called once again, this is because a new value has been assigned to the name and age of the student. On line 31, after it was executed, nothing was printed out. This is because student3 was only made without any other values. But on line 32, there was also nothing called. This is because student3 assigned the values inside the student2. After these codes the objects were destroyed by the destruct function, it destroyed the three objects that were made.

Table 2-1. Initial Driver Program

Screenshot



```cpp
#include <iostream>
class Student{
  private:
    std::string studentName;
    int studentAge;
  public:
  //constructor
    Student(std::string newName = "John Doe", int newAge=18){
      studentName = std::move(newName);
      studentAge = newAge;
      std::cout << "Constructor called" << std::endl;
    };
    ~Student(){
      std::cout << "Destructor called" <<std::endl;
    }
    //Copy Constructor
    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }
    //Display Attributes
    void printDetails(){
        std::cout<<this->studentName << " " << this->studentAge << std::endl;
    }
};
int main() {
    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

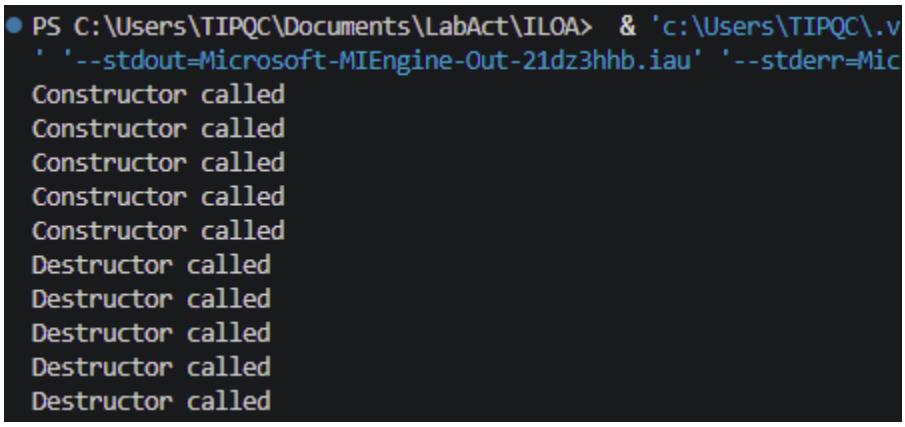
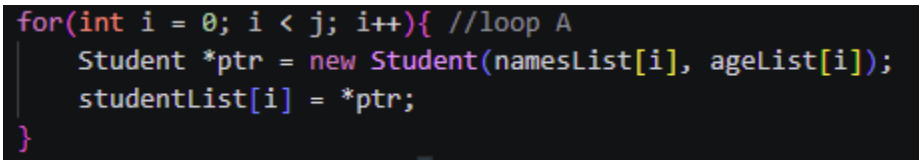
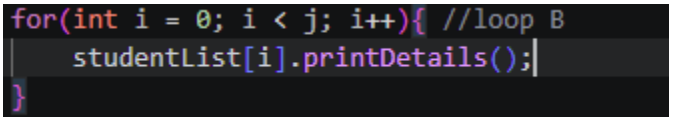
Output	
Observation	In this code, there are five names and five ages. The constructor was called five times because each of them had different values and did not copy from other objects. After the assigning of values, and all of the codes were executed successfully the destruct function was called destroying all five objects.

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	In this loop, the values were added calling the constructor and a heap has also been made and this is where the values assigned in this loop go.
Loop B	
Observation	This loop is responsible for printing the values from the array named studentList

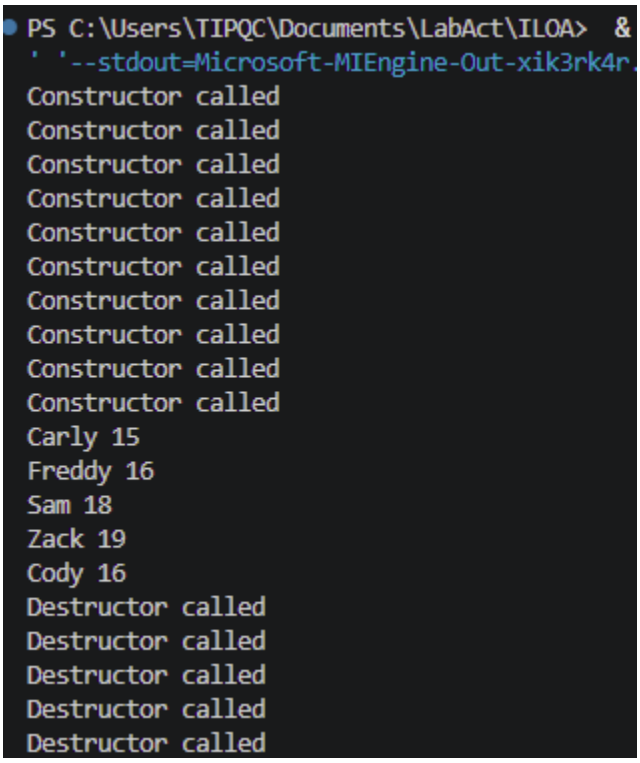
Output	
Observation	In this modified code, we can see that the notification “Construct Called” ten times, this is because before the for loops, there are five objects with different values. After line 32, another five values were assigned there for the constructor was called again five times. The second for loop allows the user to print the values that were assigned on the heap. The constructor was called ten times but the destructor was only called five times, this is because of the heap. There were five values assigned in the heap and the new C++ will not be able to destruct these values.

Table 2-3. Final Driver Program

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem1:

ILO-C.cpp | c_cpp_properties.json | CMakeLists.txt

```cpp
#include <iostream>
#include <string>

//Problem 1
class Fruits {
private:
    std::string name;
    int amount;
    int price;

public:
    // Constructor
    Fruits(std::string newName, int newAmount, int newPrice) {
        name = newName;
        amount = newAmount;
        price = newPrice;
    }

    // Destructor
    ~Fruits() {

    }

    // Copy Constructor
    Fruits(const Fruits &copyFruits) {
        name = copyFruits.name;
        amount = copyFruits.amount;
        price = copyFruits.price;
    }
```

```cpp
    // Copy Assignment Operator
    Fruits& operator=(const Fruits &copyFruits) {
        if (this != &copyFruits) {
            name = copyFruits.name;
            amount = copyFruits.amount;
            price = copyFruits.price;
        }
        return *this;
    }
};

```

```cpp
class Vegetables {
private:
    std::string name;
    int amount;
    int price;

public:
    // Constructor
    Vegetables(std::string newName, int newAmount, int newPrice) {
        name = newName;
        amount = newAmount;
        price = newPrice;
    }

    // Destructor
    ~Vegetables() {
        std::cout << "Vegetable \"" << name << "\" destroyed" << std::endl;
    }

    // Copy Constructor
    Vegetables(const Vegetables &copyVeggies) {
        name = copyVeggies.name;
        amount = copyVeggies.amount;
        price = copyVeggies.price;
        std::cout << "Vegetable copy constructor called for " << name << std::endl;
    }
```

```
69      // Copy Assignment Operator
70      Vegetables& operator=(const Vegetables &copyVeggies) {
71          if (this != &copyVeggies) {
72              name = copyVeggies.name;
73              amount = copyVeggies.amount;
74              price = copyVeggies.price;
75          }
76          return *this;
77      }
```

Problem 2:

```
int main() {
    //Problem 2
    void* GroceryList[4];

    GroceryList[0] = new Fruits( newName: "Apple", newAmount: 7, newPrice: 10);
    GroceryList[1] = new Fruits( newName: "Banana", newAmount: 8, newPrice: 10);
    GroceryList[2] = new Vegetables( newName: "Broccoli", newAmount: 12, newPrice: 60);
    GroceryList[3] = new Vegetables( newName: "Lettuce", newAmount: 10, newPrice: 50);

    std::cout << "Grocery List" << std::endl;
    static_cast<Fruits*>(GroceryList[0])->displayDetails();
    static_cast<Fruits*>(GroceryList[1])->displayDetails();
    static_cast<Vegetables*>(GroceryList[2])->displayDetails();
    static_cast<Vegetables*>(GroceryList[3])->displayDetails();
```

Problem 3:

```
// Problem 3
int TotalPrice(void* GroceryList[]) {
    int total = 0;

    if (GroceryList[0] != nullptr) total += static_cast<Fruits*>(GroceryList[0])->calculateSum();
    if (GroceryList[1] != nullptr) total += static_cast<Fruits*>(GroceryList[1])->calculateSum();

    if (GroceryList[2] != nullptr) total += static_cast<Vegetables*>(GroceryList[2])->calculateSum();
    if (GroceryList[3] != nullptr) total += static_cast<Vegetables*>(GroceryList[3])->calculateSum();

    return total;
}
```

```
    //Problem 3
     int sum = TotalPrice(GroceryList);
     std::cout << "Total: " << TotalPrice << "\n" << std::endl;
```

Modified Fruits:

```
class Fruits {
private:
    std::string name;
    int amount;
    int price;

public:
    // Constructor
    Fruits(std::string newName, int newAmount, int newPrice) {
        name = newName;
        amount = newAmount;
        price = newPrice;
    }

    // Destructor
    ~Fruits() {
    }
    // Copy Constructor
    Fruits(const Fruits &copyFruits) {
        name = copyFruits.name;
        amount = copyFruits.amount;
        price = copyFruits.price;
    }

    // Copy Assignment Operator
    Fruits& operator=(const Fruits &copyFruits) {
        if (this != &copyFruits) {
            name = copyFruits.name;
```

```
    // Copy Assignment Operator
    Fruits& operator=(const Fruits &copyFruits) {
        if (this != &copyFruits) {
            name = copyFruits.name;
            amount = copyFruits.amount;
            price = copyFruits.price;
        }
        return *this;
    }
    int calculateSum() const {
        return amount * price;
    }

    void displayDetails() const {
        std::cout << "[Fruit] Name: " << name << ", Amount: " << amount
                  << ", Price: " << price << ", Total Cost: " << calculateSum() << std::endl;
    }

    std::string getName() const { return name; }
};
```

Modified Vegetables:

```cpp
class Vegetables {
private:
    std::string name;
    int amount;
    int price;

public:
    // Constructor
    Vegetables(std::string newName, int newAmount, int newPrice) {
        name = newName;
        amount = newAmount;
        price = newPrice;
    }

    // Destructor
    ~Vegetables() {
        std::cout << "Vegetable " << name << " destroyed" << std::endl;
    }

    // Copy Constructor
    Vegetables(const Vegetables &copyVeggies) {
        name = copyVeggies.name;
        amount = copyVeggies.amount;
        price = copyVeggies.price;
        std::cout << "Vegetable copy constructor called for " << name << std::endl;
    }

    // Copy Assignment Operator
    Vegetables& operator=(const Vegetables &copyVeggies) {
        if (this != &copyVeggies) {
            name = copyVeggies.name;
            amount = copyVeggies.amount;
            price = copyVeggies.price;
        }
        return *this;
    }

    int calculateSum() const {
        return amount * price;
    }

    void displayDetails() const {
        std::cout << "[Vegetable] Name: " << name << ", Amount: " << amount
                  << ", Price: " << price << ", Total Cost: " << calculateSum() << std::endl;
    }

    std::string getName() const { return name; }
};
```

Problem 4:

```
    // Problem 4

    if (GroceryList[3] != nullptr) {
        Vegetables* lettucePtr = static_cast<Vegetables*>(GroceryList[3]);
        delete lettucePtr;
        GroceryList[3] = nullptr;
    }

    return 0;
}
```

Output:

```
"C:\Users\Vince Urbano\Documents\ILO-C_HOA2\cmake-build-debug\ILO_C_HOA2.exe"
Grocery List
[Fruit] Name: Apple, Amount: 7, Price: 10, Total Cost: 70
[Fruit] Name: Banana, Amount: 8, Price: 10, Total Cost: 80
[Vegetable] Name: Broccoli, Amount: 12, Price: 60, Total Cost: 720
[Vegetable] Name: Lettuce, Amount: 10, Price: 50, Total Cost: 500
Total: 1370

Vegetable Lettuce destroyed

Process finished with exit code 0
```

Code Analysis:

In this code, I have created two main classes named Fruits and Vegetables. In these classes they both have a constructor, destructor, copy constructor and the copy assignment constructor. The constructor will be responsible for assigning values such as the name, amount and the price of the object. The destructor will be executed once the code has been finished, it will be responsible for deleting the values at the end of the program. A copy constructor is used to create an object that is exactly alike from the object. Copy assignment will be used for copying values from the other objects, the if statement is used to check if the new value is the same as the current value. The function calculateSum is responsible for getting the total price of that specific class. Moving to the int main(), we have used void*GroceryList because we need both of the classes to appear on the array that we have made and because of this, we have used static_cast. The static_cast is responsible to tell the program which specific class it belongs to. The function of TotalPrice is used to get the total prices from the grocery list. Moving to problem 4, we have used another if statement to check if the lettuce is still on the array, if it is not the program will delete it and use a nullptr which shows that another object can be added to it again anytime soon.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation for the function of heap and static_cast.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used the Gemini AI to ask how a heap and static_cast works and how it also affects the codes.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.